**Home Assignment:**

1.Prepare a test strategy document based on the domain, using a chain-of-thought approach

Context: Act like a Senior QA Lead and prepare the document

Instruction:

Generate test strategy document for an Enterprise application in Energy and Utiities Domain

[IMPORTANT] Generate the document for testing of electricity distribution related modules

-ASK me 3 questions one by one for this document creation

-Ask me the next question after me providing the answer

-Document should be Entrperise following the industry standard

Output

[STRICT] Generate document .Add the review comments separately

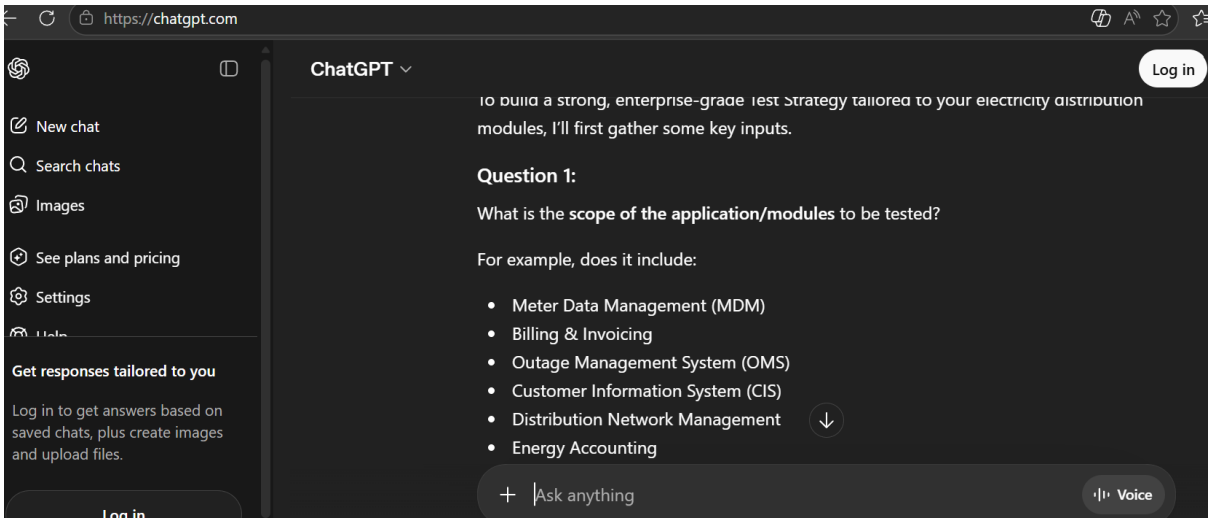
as a table format for generated document

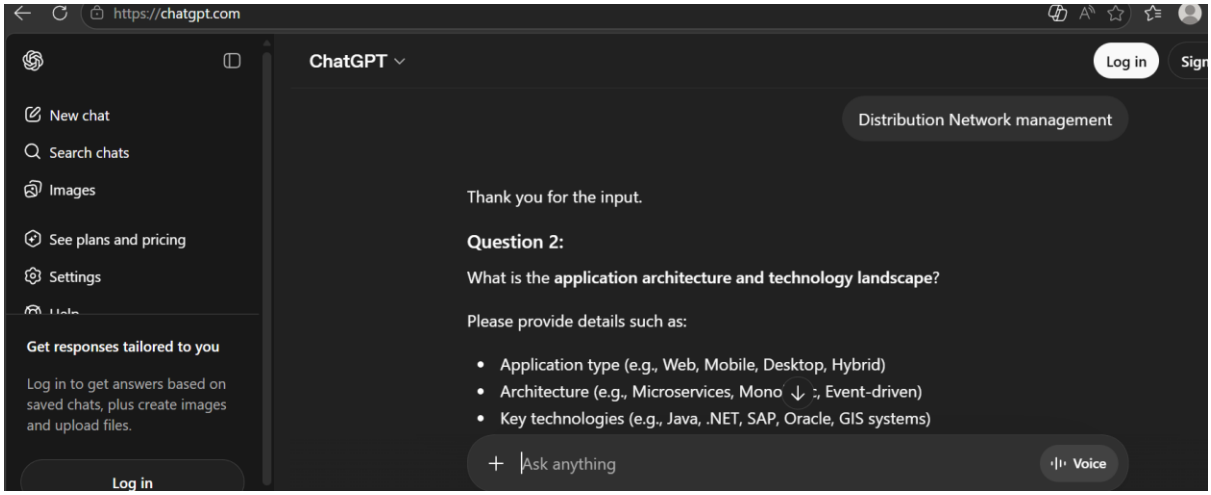
Tone :

Give the output in Professional tone only

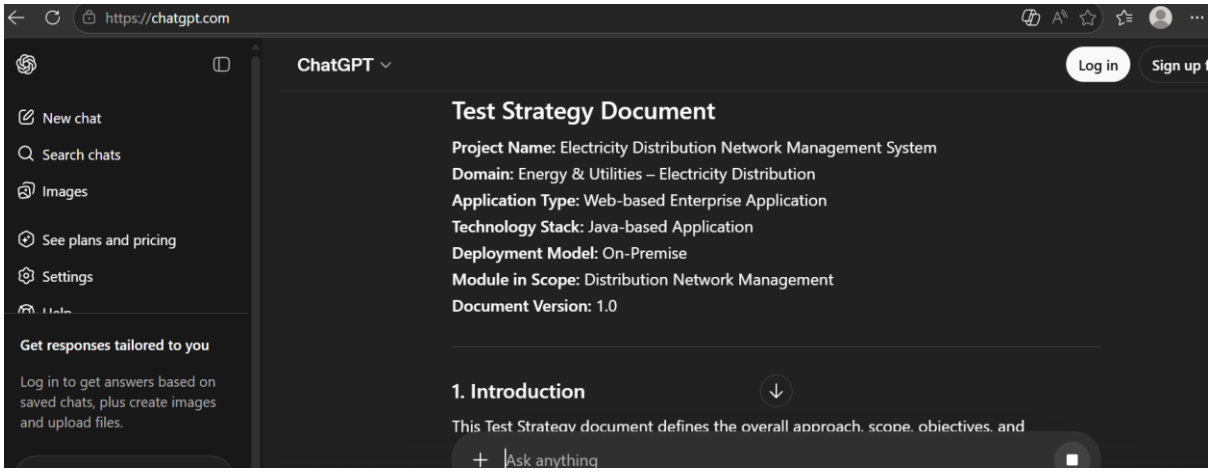
Persona:

Review the Test Strategy document as a Domain Expertise QA manager and give the review comments

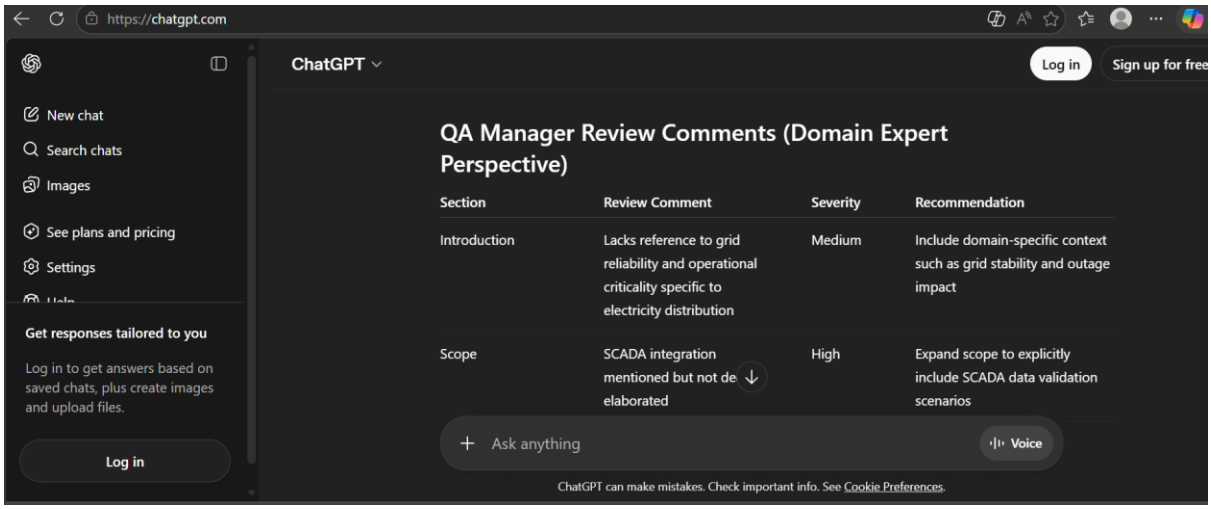




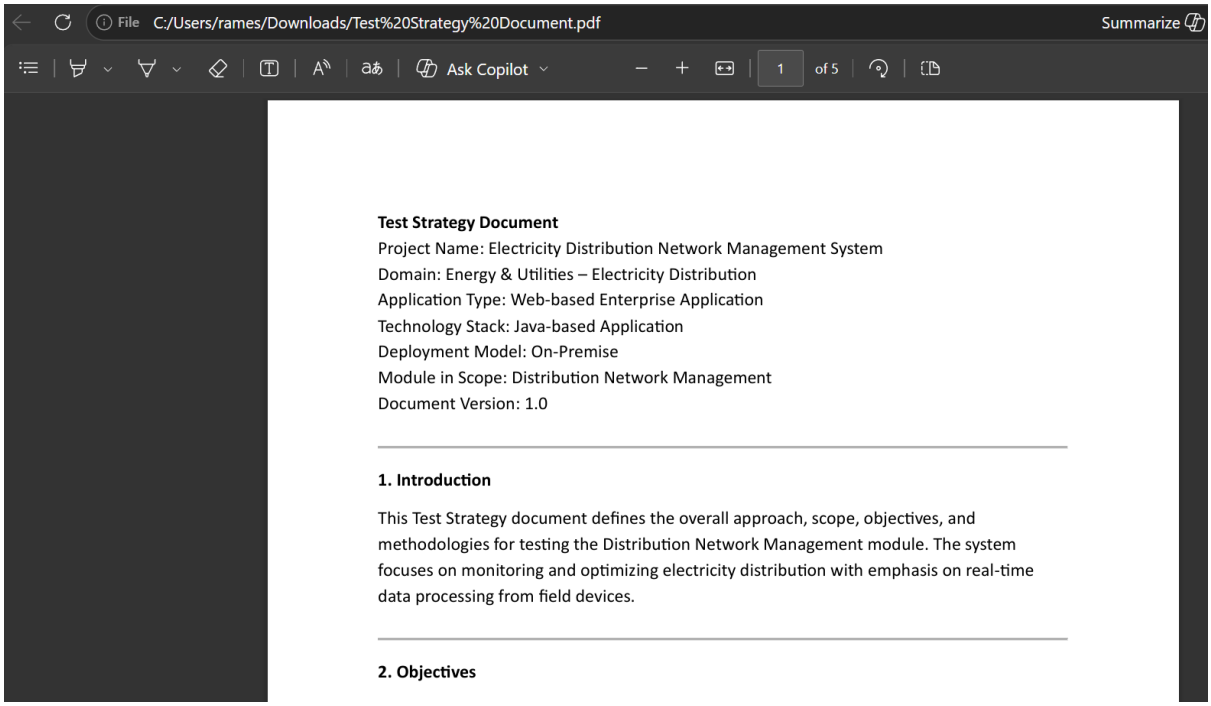

Answered all 3 questions and then the Test Strategy document is generated.



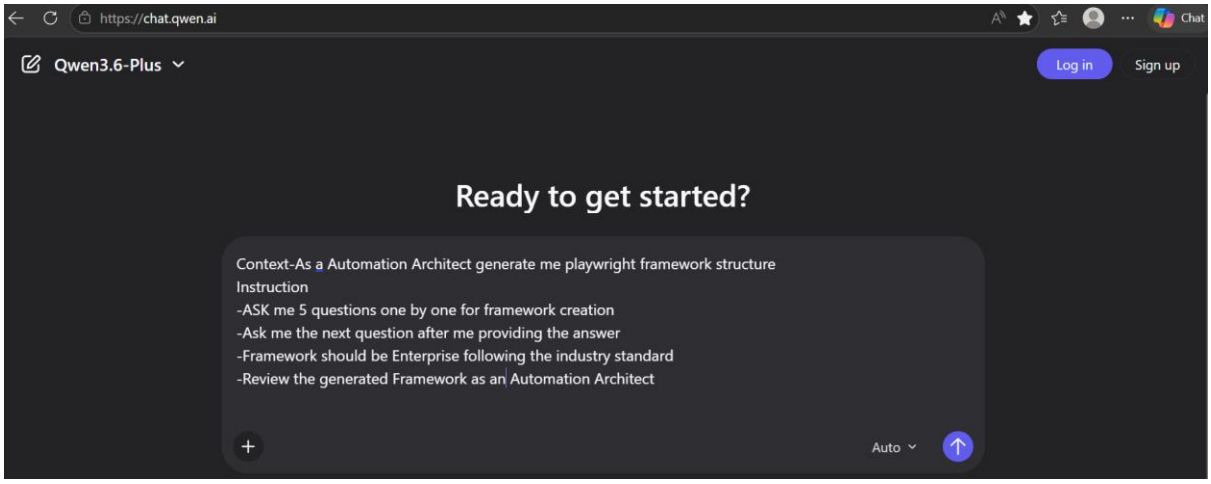

Review comment



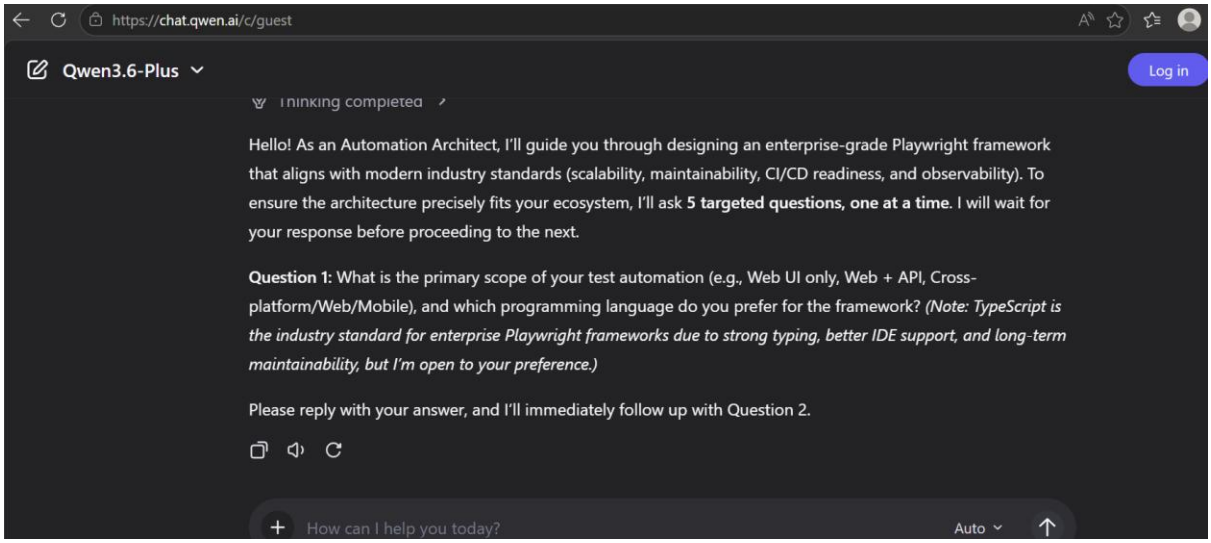

Downloaded



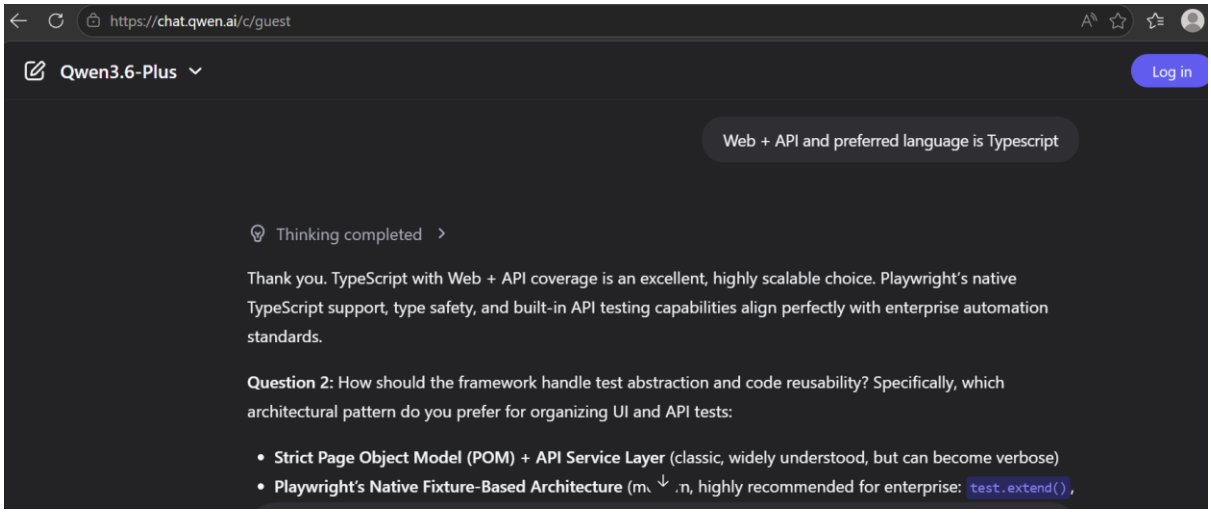

2. Design the automation framework structure using a chain-of-thought approach



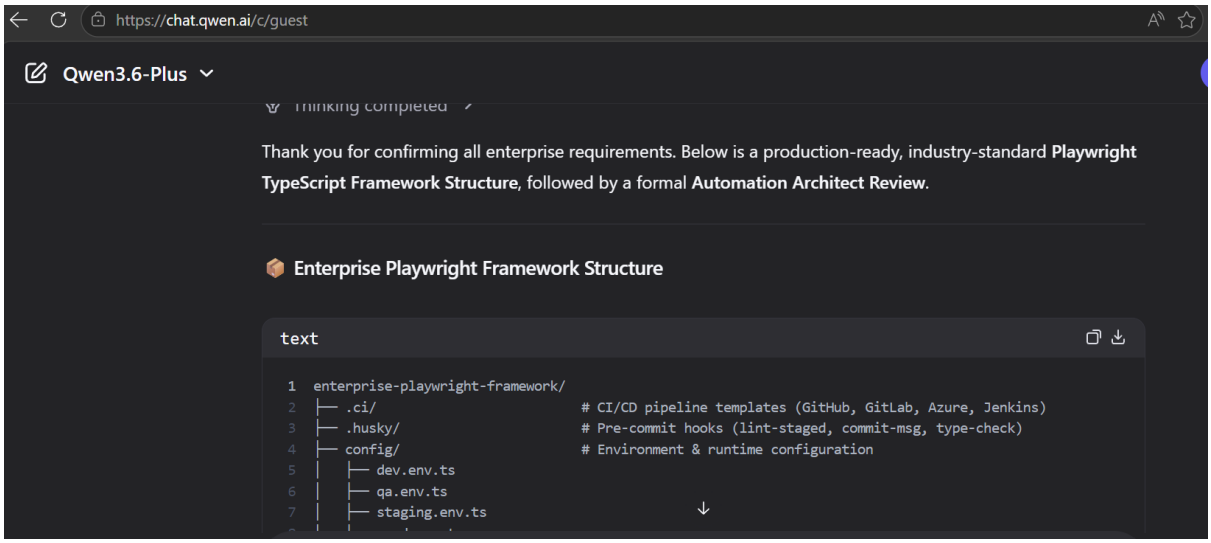

As part of chain of thoughts, we are getting the questions as below,



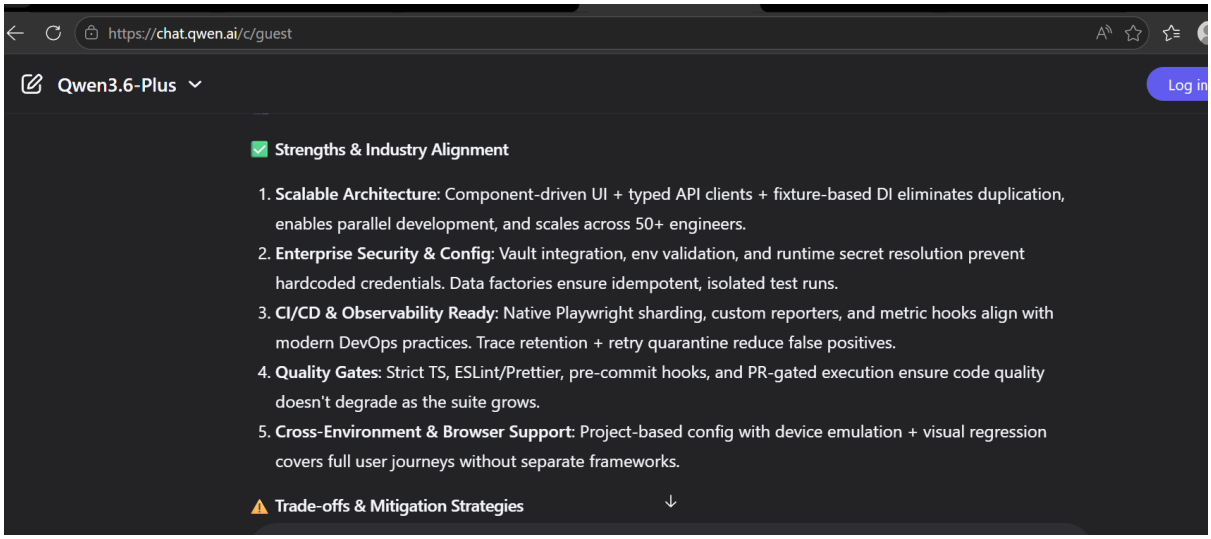

Answering them




Continuing with the remaining and finally the framework created



**Framework Structure:**

```
enterprise-playwright-framework/
├── .ci/                    # CI/CD pipeline templates (GitHub, GitLab, Azure, Jenkins)
├── .husky/                 # Pre-commit hooks (lint-staged, commit-msg, type-check)
├── config/                 # Environment & runtime configuration
│   ├── dev.env.ts
│   ├── qa.env.ts
│   ├── staging.env.ts
│   ├── prod.env.ts
│   └── vault.manager.ts        # AWS Secrets Manager / Azure Key Vault / HashiCorp adapter
├── src/                    # Core framework & reusable layer
│   ├── core/               # Foundation & cross-cutting concerns
│   │   ├── fixtures/           # Base test.extend() (DI-style dependencies, auth, context)
│   │   ├── helpers/            # Retry logic, custom assertions, formatters, wait strategies
│   │   ├── reporters/          # Playwright reporter hooks (Allure, HTML, Datadog/Splunk, Slack)
│   │   └── validators/         # Schema validation (Zod/Yup), config sanity checks
│   ├── api/                # API testing layer
│   │   ├── clients/            # Typed APIRequestContext wrappers, interceptors, DTO mappers
│   │   ├── factories/          # Dynamic data generation (CRUD + auto-cleanup fixtures)
│   │   └── fixtures/           # API-specific test fixtures
│   └── ui/                 # UI testing layer
│       ├── components/         # Atomic, reusable UI components (Table, Modal, Form, Toast)
│       ├── pages/              # Page wrappers composing components + domain logic
│       └── fixtures/           # UI state fixtures (login, navigation, feature flags)
├── tests/                  # Test implementation (tagged, sharded, domain-driven)
│   ├── e2e/                # Cross-API+Web user journeys
│   ├── api/                # Contract, integration, & error-boundary tests
│   ├── visual/             # Snapshot + Applitools/Percy visual regression suites
│   └── smoke/              # Fast PR-validation pipelines (<5 min)
├── docs/                   # Architecture Decision Records (ADRs), runbooks, onboarding
```

```
├── playwright.config.ts      # Centralized execution & project config
├── package.json              # Dependencies, scripts, workspaces (if monorepo)
├── tsconfig.json             # Strict TS, path aliases, module resolution
├── .eslintrc.cjs             # Strict ESLint + Playwright-specific rules
├── .prettierrc               # Formatting standards
├── .env.example              # Secret placeholders
├── Dockerfile                # Containerized runner (Node + Playwright browsers)
└── README.md                 # Setup, execution, contribution guidelines
```